MODULE - 01 :-

① Reduce the following boolean expressions to the minimum number of literals.

=> i) $x(x'+y)$

$x \cdot x' + xy$

$0 + xy$

$x \cdot y$

ii) $x + x'y$

$(x+x')(x+y)$

$1 \cdot (x+y)$

$x+y$

iii) $(x+y)(x+y')$

$xx + xy' + xy + yy'$

$x + xy' + xy + 0$

$x(1+y'+y)$

$x(1+0)yy')$

$x(1+0)$

x //

iv) $xy + x'z + yz$ (by consensus theorem)

$xy + x'z + yz(x+x')$

$xy + x'z + xyz + x'yz$

by duality theorm.

$xy(1+z) + x'z(1+y)$

$xy + x'z$

⑤ $(x+y)(x'+z)(y+z)$ => by duality theorm

$(x+y)(x'+z)$ //

② Find the compliment of a function.

i) $F = x'yz' + x'y'z$

$F' = (x'yz' + x'y'z)'$

$= (x'yz')' \cdot (x'y'z)'$

$= (x+y'+z)(x+y+z')$ //

$(ABC)' = A'+B'+C'$

ii) $F = x(y'z + yz)$

$F' = (x \cdot (y'z + yz))'$

$= x' + (y'z + yz)'$

$= x' + (y'z)'(yz)'$

$= x' + (y+z')(y'+z')$ //

iii) $F = (xy' + x'y)$

$F' = (xy' + x'y)'$

$= (xy')' + (x'y)'$

$= (x'+y) \cdot (x+y')$

$(A+B)' = A' \cdot B'$

$(A \cdot B)' = A' + B'$

iv) $F = wx + yz$

$F' = (wx+yz)'$

$= (wx)' + (yz)'$

$= (w'+x').(y'+z')$ //.

P.T. $F.F' = 0$ & $F + F' = 1$ $\quad F = wx+yz$

$F = wx+yz$

$F' = (wx+yz)'$

$F' = (w'+x').(y'+z')$

$F.F' = 0$.

LHS $= (wx+yz).(w'+x')(y'+z')$

$= (ww'x + wxx' + w'yz + x'yz)(wxy' + wxz' + yy'z + yzz')$

$= 0$.

③ Find complement of fun by taking duality & complementing each literal.

=> i) $F = x'.y.z' + x'.y'.z$

Duality of

$F = (x'+y+z').(x'+y'+z)$

Complement of each literal

$F = (x+y'+z).(x+y+z')$

ii) $F = x(y'z + yz)$

Duality of

$F = x + (y'+z).(y+z)$

Complement of each literal

$F = x' + (y+z').(y'+z')$

only Find complement.

iii) $F = z + z'(v'w + xy)$

$F' = (z + z'(v'w + xy))'$

$= z'.(z'(v'w + xy))'$

$= z'.z + (vw' + x'y')$

$= z'.z + (v+w')(x'+y')$

Σm - SOP

④ Describe map method for.

MAP METHOD :- It is also known as K-MAP (or) Karnaugh MAP.

It is made up of square with each square representing 1 minterm of the fun this is to be minimize.

i) TWO-VARIABLE K-MAP :-

x \ y	0	1
0	$\bar{x}\bar{y}^0$	$\bar{x}y^1$
1	$x\bar{y}^2$	xy^3

$M_0 \rightarrow 00$
$M_1 \rightarrow 01$
$M_2 \rightarrow 10$
$M_3 \rightarrow 11$

there are 4 minterms for 2 variables.
Hence, map consists of 4 squares, 1 ~~ter~~ for each minterm.

ii) THREE-VARIABLE K-MAP

there are 8 minterms for 3 binary variables

∴ map consists of 3 variables.

x \ yz	00	01	11	10
0	$\bar{x}\bar{y}\bar{z}^0$	$\bar{x}\bar{y}z^1$	$\bar{x}yz^3$	$\bar{x}y\bar{z}^2$
1	$x\bar{y}\bar{z}^4$	$x\bar{y}z^5$	xyz^7	$xy\bar{z}^6$

$M_0 \rightarrow 000$
$M_1 \rightarrow 001$
$M_2 \rightarrow 010$
$M_3 \rightarrow 011$
$M_4 \rightarrow 100$
$M_5 \rightarrow 101$
$M_6 \rightarrow 110$
$M_7 \rightarrow 111$

iii) FOUR-VARIABLE K-MAP :-

there are 16 minterms for 4-variable K-MAP

∴ MAP consists of 16 squares

wx \ yz	00	01	11	10
00	$\bar{w}\bar{x}\bar{y}\bar{z}^0$	$\bar{w}\bar{x}\bar{y}z^1$	$\bar{w}\bar{x}yz^3$	$\bar{w}\bar{x}y\bar{z}^2$
01	$\bar{w}x\bar{y}\bar{z}^4$	$\bar{w}x\bar{y}z^5$	$\bar{w}xyz^7$	$\bar{w}xy\bar{z}^6$
11	$wx\bar{y}\bar{z}^{12}$	$wx\bar{y}z^{13}$	$wxyz^{15}$	$wxy\bar{z}^{14}$
10	$w\bar{x}\bar{y}\bar{z}^8$	$w\bar{x}\bar{y}z^9$	$w\bar{x}yz^{11}$	$w\bar{x}y\bar{z}^{10}$

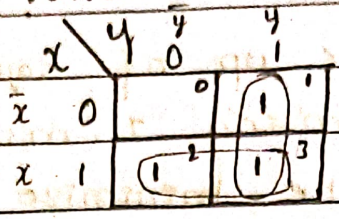
$M_0 \rightarrow 0000$
$M_1 \rightarrow 0001$
$M_2 \rightarrow 0010$
$M_3 \rightarrow 0011$
$M_4 \rightarrow 0100$
$M_5 \rightarrow 0101$
$M_6 \rightarrow 0110$
$M_7 \rightarrow 0111$
$M_8 \rightarrow 1000$
$M_9 \rightarrow 1001$
$M_{10} \rightarrow 1010$
$M_{11} \rightarrow 1011$
$M_{12} \rightarrow 1100$
$M_{13} \rightarrow 1101$
$M_{14} \rightarrow 1110$

$F(a,b,c,d) = M_0 + M_2 + M_5 + M_6.$

⑤ Apply K-MAP technique (or) simplify the following expressions by using K-MAP.

=> 1) $F(x,y) = \sum(1,2,3)$

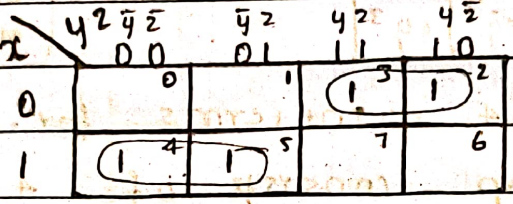
by 2-variable K-MAP



$F = x + y$.

2) $F = (x,y,z) = \sum(2,3,4,5)$

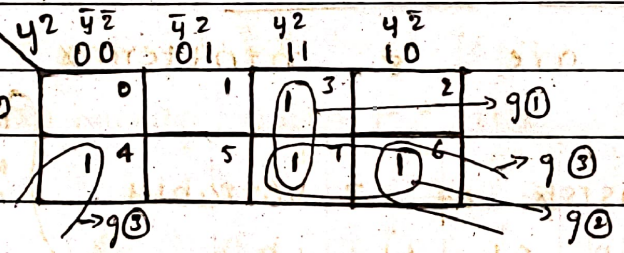
by 3-variable K-MAP



$F = x\bar{y} + \bar{x}y$

POS =

3) $F(x,y,z) = \sum(3,4,6,7)$



Here group ② is covered by group ① & ③

∴ it will be redendent group so, we eleminate g②

$F = yz + x\bar{z}$

4) $F = \bar{A}C + \bar{A}B + A\bar{B}C + BC$

=> $F = \bar{A}\blacksquare C + \bar{A}B\blacksquare + A\bar{B}C + \blacksquare BC$

$F = \bar{A}C(B+\bar{B}) + \bar{A}B(C+\bar{C}) + A\bar{B}C + (A+\bar{A})BC$

$F = \underline{\bar{A}BC} + \bar{A}\bar{B}C + \underline{\bar{A}BC} + \bar{A}B\bar{C} + \underline{A\bar{B}C} + \underline{ABC} + \underline{\bar{A}BC}$

$= \bar{A}BC + \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}C + ABC$

011 (3) 001 (1) 010 (2) 101 (5) 111 (7)

∴ $F = \sum(3,1,2,5,7)$

DATE / /
PAGE NO

In POS (or) SOP (or) D

$x = 0$, $\sum = 1$, $d = X$

A \ BC	00	01	11	10
$\bar{A}$	0	1	3	2
A	4	5	7	6

$F = \cancel{z + \bar{x}y}$ $\quad F = C + \bar{A}B$

5) $F(x,y,z) = \Sigma(0,2,4,5,6)$

6) $F(x,y,z) = x'y + yz' + y'z'$

7) $F(w,x,y,z) = \Sigma(0,1,2,4,5,6,8,9,12,13,14)$

wx \ yz	00	01	11	10
$\bar{w}\bar{x}$	0	1	3	2
$\bar{w}x$	4	5	7	6
wx	12	13	15	14
$w\bar{x}$	8	9	11	10

$F = \bar{y} + x\bar{z} + \bar{w}\bar{z}$

$F = y$

8) $F = \bar{A}\bar{B}\bar{C} + \bar{B}C\bar{D} + \bar{A}BC\bar{D} + A\bar{B}\bar{C}$

$\Rightarrow F = \bar{A}\bar{B}\bar{C}(D+\bar{D}) + (A+\bar{A})BC\bar{D} + \bar{A}BC\bar{D} + A\bar{B}\bar{C}(D+\bar{D})$

$F = \underset{0001}{\bar{A}\bar{B}\bar{C}D} + \underset{0000}{\bar{A}\bar{B}\bar{C}\bar{D}} + \underset{1010}{A\bar{B}C\bar{D}} + \underset{0010}{\bar{A}\bar{B}C\bar{D}} + \underset{0110}{\bar{A}BC\bar{D}} + \underset{1001}{A\bar{B}\bar{C}D} + \underset{1000}{A\bar{B}\bar{C}\bar{D}}$

(1) (0) (10) (2) (6) (9) (8)

$F = \Sigma F(A,B,C,D) = \Sigma(1,0,10,2,6,9,8)$

AB \ CD	00	01	11	10
$\bar{A}\bar{B}$	0	1	3	2
$\bar{A}B$	4	5	7	6
AB	12	13	15	14
$A\bar{B}$	8	9	11	10

$F = \bar{B}\bar{C} + \bar{B}\bar{D} + \bar{A}C\bar{D}$

In POS form $\Rightarrow F = (\bar{B}+\bar{C}) \cdot (\bar{B}+\bar{D}) \cdot (\bar{A}+C+\bar{D})$

9) $F(A,B,C,D) = \Sigma(0,6,7,8,12,13,14,15)$

10) $F(A,B,C,D) = \Sigma(3,4,6,7,12,13,14,15,11)$

11) $F(w,x,y,z) = \Sigma(0,1,2,5,8,9,10)$

12) $F(w,x,y,z) = x'z' + wyz + w'y'z' + x'y.$

⑥ DON'T CARE CONDITION

the unspecified minitcrms of a function are called don't care condition.

Problems:

1) simplify the boolean function $F(w,x,y,z) = \Sigma(1,3,7,11,15)$ which has the don't care conditions $d(w,x,y,z) = \Sigma(0,2,5)$ by 4 variable K-MAP

wx \ yz	$\bar{y}\bar{z}$ 00	$\bar{y}z$ 01	yz 11	$y\bar{z}$ 10
$\bar{w}\bar{x}$ 00	X (0)	1 (1)	1 (3)	X (2)
$\bar{w}x$ 01	(4)	X (5)	1 (7)	(6)
wx 11	(12)	(13)	1 (15)	(14)
$w\bar{x}$ 10	(8)	(9)	1 (11)	(10)

$F = yz + \bar{w}\bar{x}$

2) $F(x,y,z) = \Sigma(0,2,4,6)$ & $d(x,y,z) = \Sigma(1,5)$

x \ yz	$\bar{y}\bar{z}$ 00	$\bar{y}z$ 01	yz 11	$y\bar{z}$ 10
$\bar{x}$ 0	1 (0)	X (1)	(3)	1 (2)
x 1	1 (4)	X (5)	(7)	1 (6)

$F = \bar{z}$

3) $F(x,y,z) = \pi(2,3,4,5)$ & $d(x,y,z) = \pi(6,7)$

x \ yz	$\bar{y}\bar{z}$ 00	$\bar{y}z$ 01	$\bar{y}\bar{z}$ 11	$\bar{y}z$ 10
$\bar{x}$ 0	(0)	(1)	0 (3)	0 (2)
$\bar{x}$ 1	0 (4)	0 (5)	X (7)	X (6)

$F = \bar{x} + \bar{y}$

In POS Form :- $F = (x\bar{y}')$

In all form Σ, exp = $\bar{B}\,\bar{A}$

When $\bar{A}\,\bar{B}$

Program 2: Simulate t... following CPU scheduling algorithms to find turn ... time and waiti...

MODULE-02

of the operand ... a constant value ... register.

t of BCD [Binary c... onverter

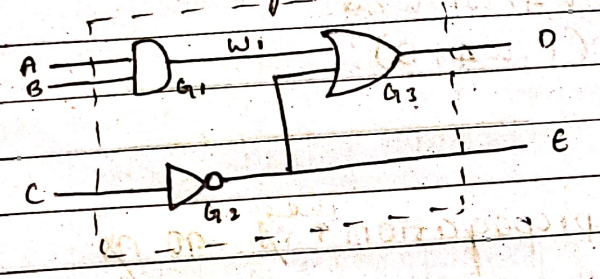
VERILOG :

A verilog model is composed of text using keyword.

Keywords are predefined lower case identifiers, "100 keywords in verilog"

Eg: module, end-module, and, i/p, o/p, or, not etc

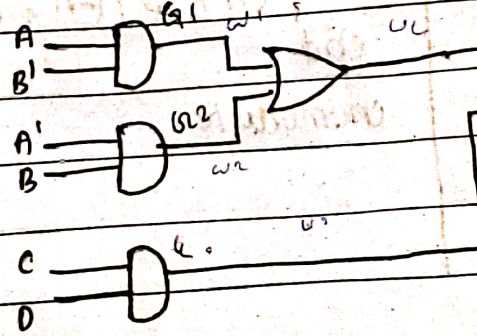
i) Develop a verilog of the circuit shown below



CODE :-

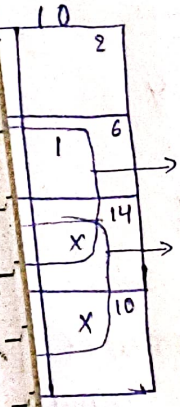
```
Module simple_circuit (A,B,C,D,E)
output D,E;
input A,B,C;
Wire W1;
and G1 (W1, A, B);
not G2 (E, C);
or G3 (D, W1);
end module
```

ii) Write verilog code & time diagram for the given circuit with propagation delay where the AND, OR gate has a delay of 30ns & 10ns.



D
0
1
0
1
0
1
0
1
0
1
0
1
0

)& d(10,11,12,13,

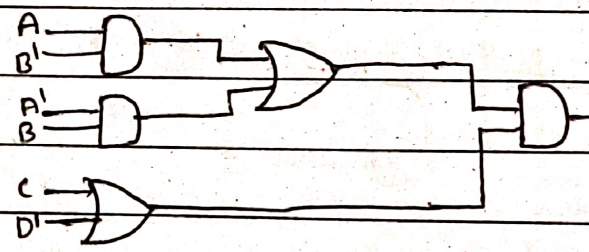


=> Verilog code :

```
module logic_circuit (A,B,C,D,F);
  input A,B,C,D;
  output F;
  wire W1,W2,W3,W4;
  and g1 (W1, A, ~B);
  and g2 (W2, ~A, B);
  and g3 (W3, C, D);
  OR g4 (W4, W1, W2, W3);
  and g5 (F, W4, D);
  end module
```

the overall propagation delay is 90 ns.

Verilog Code :



Dataflow model :

```
M c-d (A,B,C,D,F)
i/p A,B,C,D;
o/p F;
assign F = ((A&~B) | (~A&
          & (C&~D);
endmodule
```

Structural model :

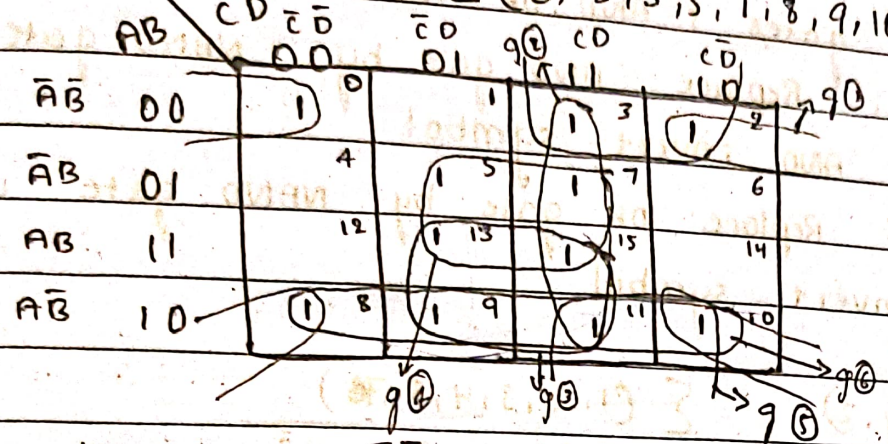
```
module circuit_st (A,B,C,D,F);
input A,B,C,D;
output F;
wire x1,x2,x3,y;
and (x1, A, ~B);
and (x2, ~A, B);
and (x3, C, ~D);
OR (y, x1, x2);
and (F, y, x3);
endmodule
```

Behavioral model :

```
module circuit_behavioral (A,B,C,D,F
input (A,B,C,D);
output reg F;
always @ (A OR B OR C OR D)
begin
F = ((A&~B | (~A & B)) & ((C&~D)
end
endmodule
```

① Find the prime implicant & essential prime implicant for the following functions.

i) $F(A,B,C,D) = \sum(0,2,3,5,7,8,9,10,11,13,15)$



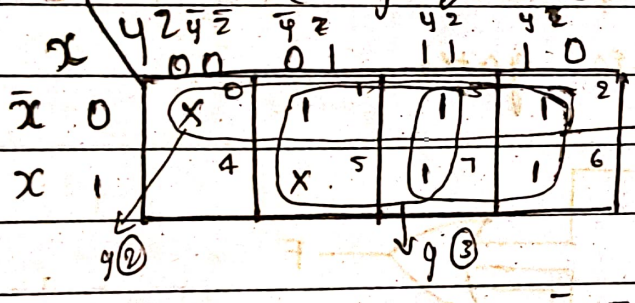
g① = $\bar{D}\bar{B}$
g② = CD
g③ = AD
g④ = BD
g⑤ = $\bar{B}C$
g⑥ = $A\bar{B}$

Prime implicants are $\bar{D}\bar{B}$, CD, AD, BD, $\bar{B}C$, $A\bar{B}$

essential prime implicants is BD, $\bar{D}\bar{B}$ //

ii) $F(x,y,z) = \sum(1,2,3,6,7)$ &

$d(x,y,z) = \sum(0,5)$

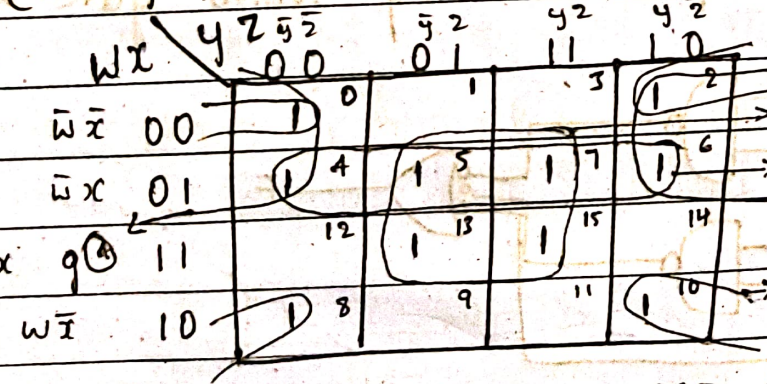


g① = y
g② = $\bar{x}$
g③ = z

Prime implicants are y $\bar{x}$ z

essential prime implicants is y //

iii) $F(w,x,y,z) = \sum(0,2,4,5,6,7,8,10,13,15)$



g① ⇒ xz
g② ⇒ $\bar{w}x$
g③ ⇒ $\bar{x}\bar{z}$
g④ ⇒ $\bar{w}\bar{z}$

Prime implicants are xz, $\bar{w}x$, $\bar{x}\bar{z}$, $\bar{w}\bar{z}$

essential prime implicant is xz, $\bar{x}\bar{z}$

iii) $F(A,B,C,D) = \sum(0,2,3,5,7,8,10,14,11,15)$

⑨ IMPLEMENT THE BOOLEAN FUNCTION USING NAND GATE :-

* STEP 01 :- Implement the function with basic gates (AND, OR & NOT)

* STEP 02 :- Replace AND gate by NAND gate with AND invert symbol.

* STEP 03 : Replace OR gate by NAND gate with invert symbol

i) $F(x,y,z) = \sum(1,2,3,4,5,7)$

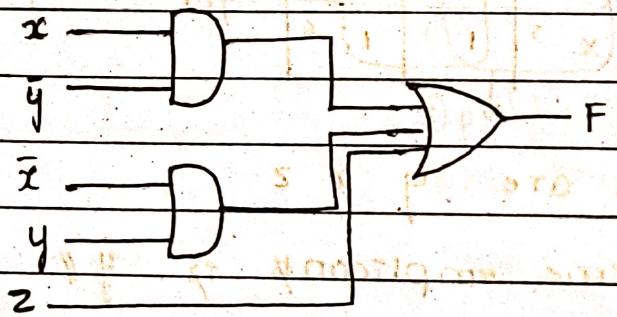
=>

x \ yz	$\bar{y}\bar{z}$ (00)	$\bar{y}z$ (01)	yz (11)	$y\bar{z}$ (10)
$\bar{x}$ 0	0	1	3	2
x 1	4	5	7	6

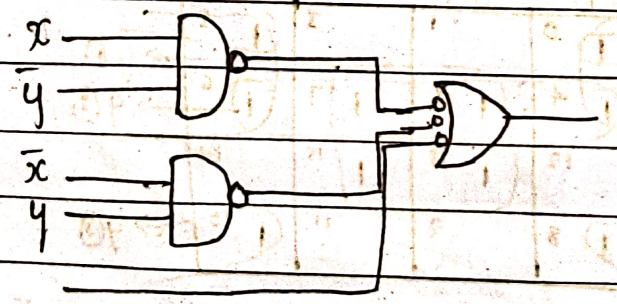
$F = x\bar{y} + \bar{x}y + z$

Implementation of NAND Gate.

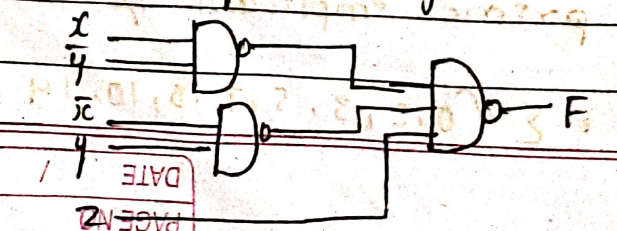
* by using basic gates.



* by using replace AND by NAND gate with invert symbol.



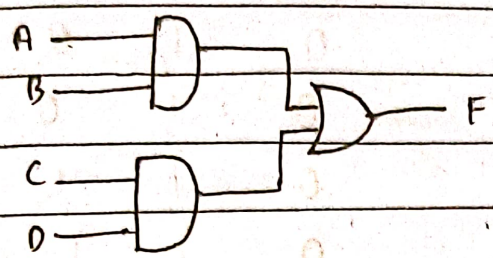
* Replace OR by NAND gate with invert symbol.



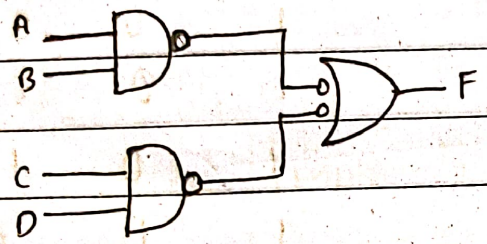
DATE / / PAGE NO

ii) $F = AB + CD$.

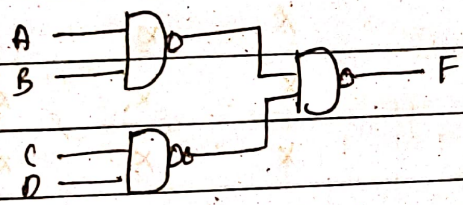
* by using basic gates



* Replace AND by NAND gate with AND invert symbol.

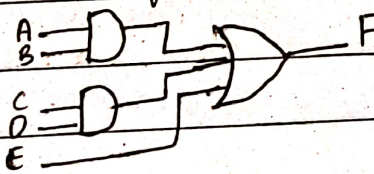


* Replace OR gate by NAND gate with invert symbol

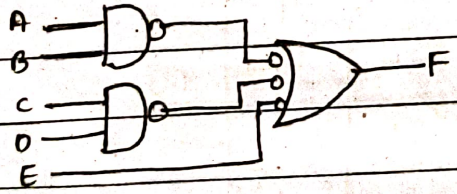


iii) $F = AB + CD + E$

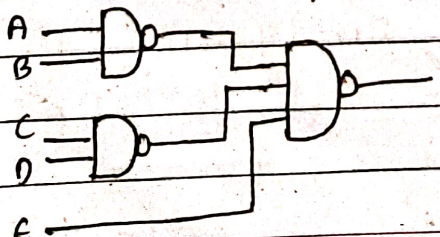
* by using basic gates



* Replace AND by NAND with AND invert symbol.



* Replace OR by NAND with invert symbol.



MODULE-02

① Implement of BCD [Binary coded decimal] to express 3 code converter

For O/P add (+3)

10 to 15 => x

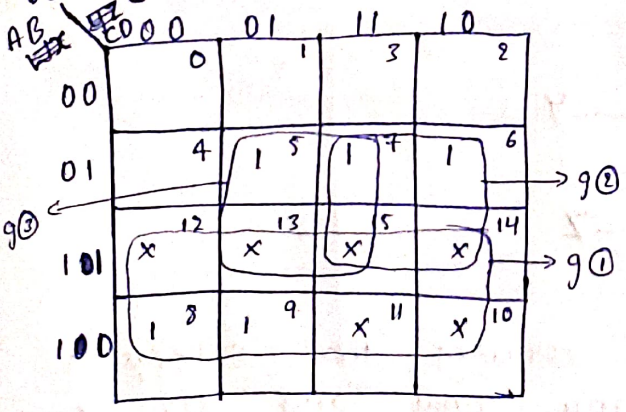
⇒ INPUTS

	A	B	C	D
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15.	1	1	1	1

OUTPUTS

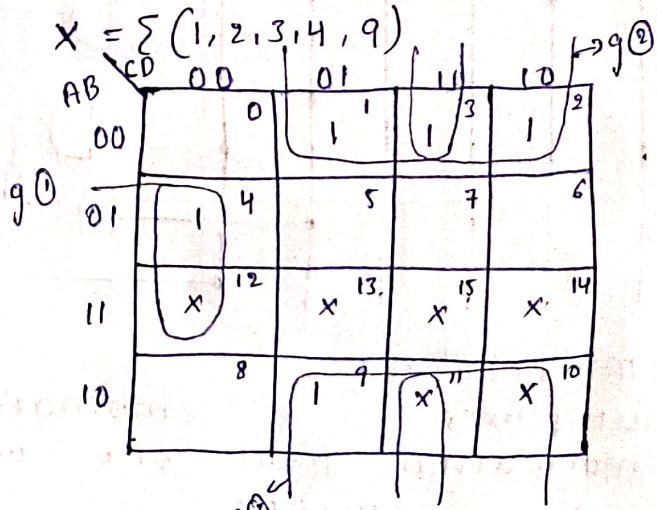
	W	X	Y	Z
0	0	0	1	1
1	0	1	0	0
2	0	1	0	1
3	0	1	1	0
4	0	1	1	1
5	1	0	0	0
6	1	0	0	1
7	1	0	1	0
8	1	0	1	1
9	1	1	0	0
10	x	x	x	x
11	x	x	x	x
12	x	x	x	x
13	x	x	x	x
14	x	x	x	x
15.	x	x	x	x

$W = \Sigma(5,6,7,8,9)$ & $d(10,11,12,13,14,15)$



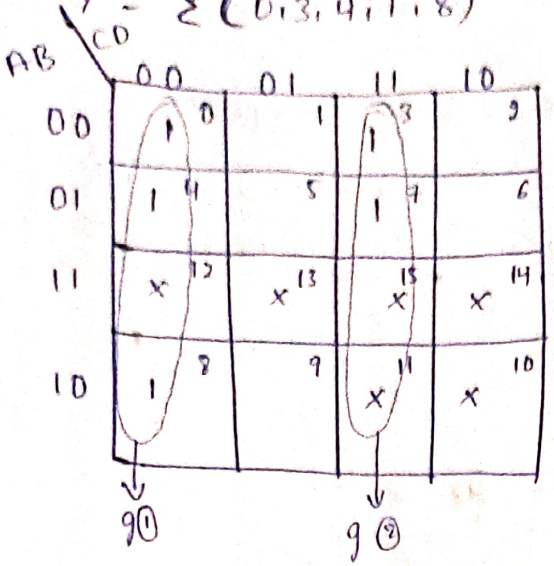
$W = A + BD + CB$

$X = \Sigma(1,2,3,4,9)$

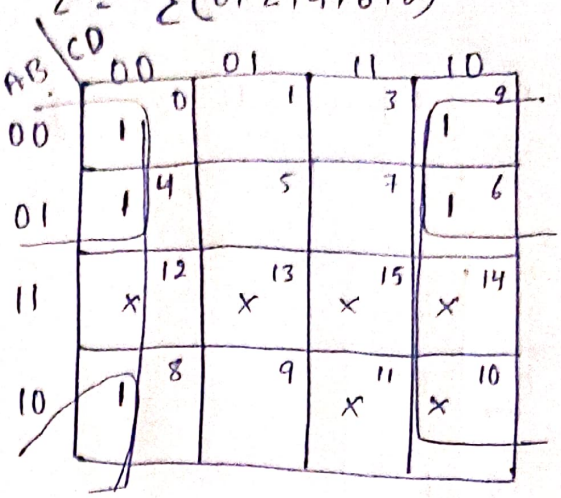


$X = BC'D' + B'C + B'D$

$Y = \Sigma(0,3,4,7,8)$



$Z = \Sigma(0,2,4,6,8)$



$Z = D'$

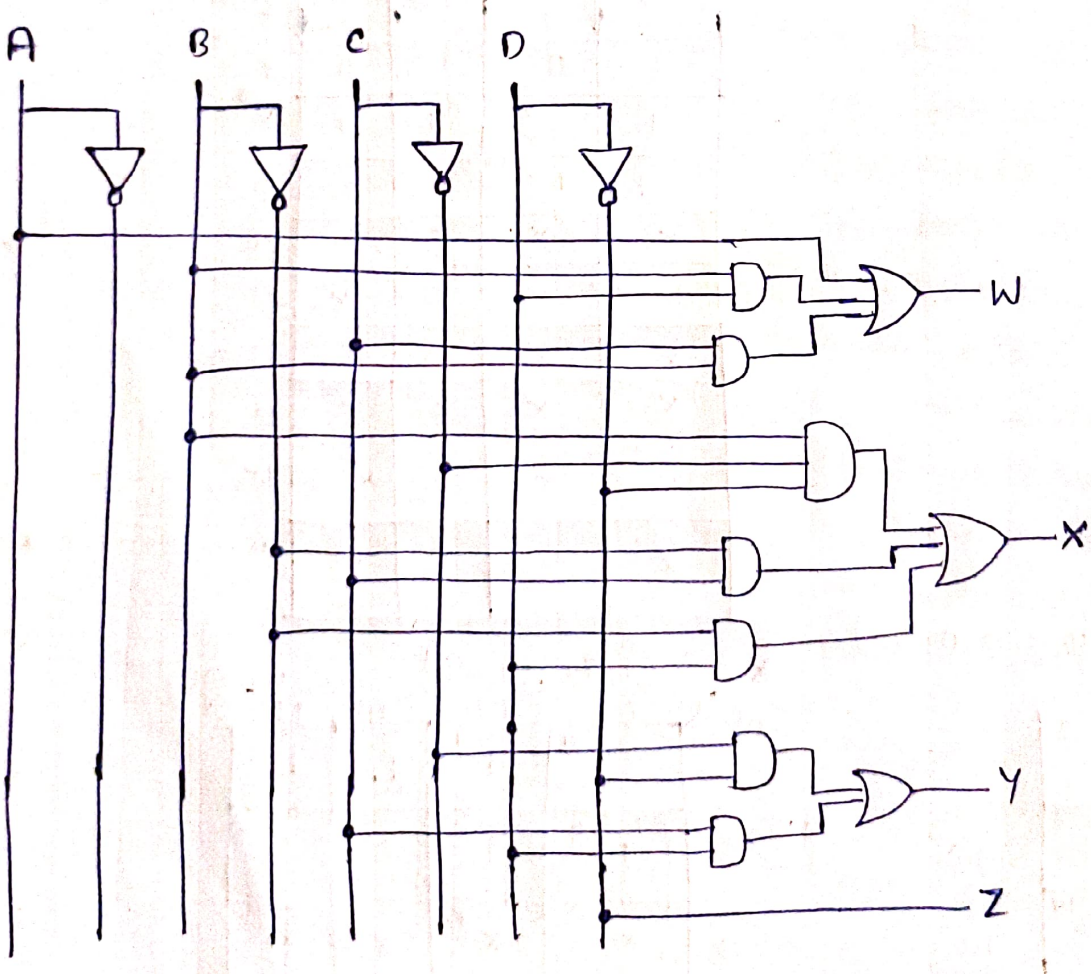
$Y = C'D' + CD$

$W = A + BD + CB$

$X = BC'D' + B'C + B'D$

$Y = C'D' + CD$

$Z = D'$



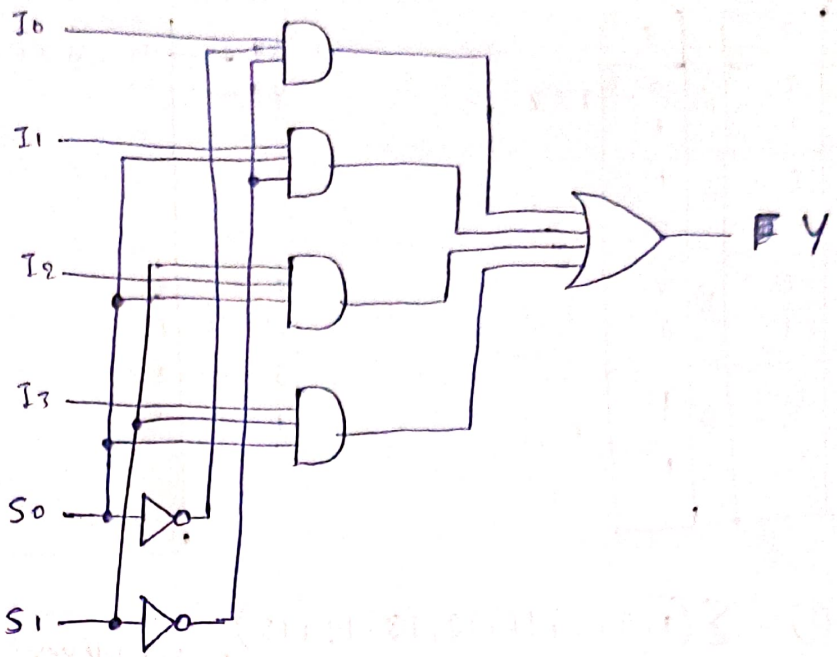
DEFINE MULTIPLUXER :

multi multipluxer is a combinational circuit that selects a binary information from one of many input and directs to single output line.

4 : 1 MUX

Truth table :

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



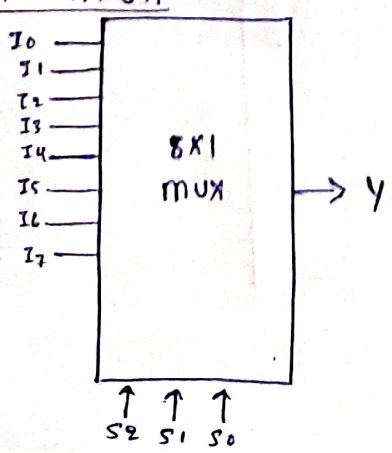
Verilog code for 4 : 1

```
module mux4to1(y, I0, I1, I2, I3, s)
output y;
input I0, I1, I2, I3;
input [1:0] s;
reg out;
always @(I0, I1, I2, I3, s)
case (s)
2'b00 : out = I0;
2'b01 : out = I1;
2'b10 : out = I2;
2'b11 : out = I3;
endcase
end module.
```

verilog code for 8 : 1

```
module mux8to1(y, I0, I1, I2, I3, I4, I5, I6, I7, s)
output y;
input I0, I1, I2, I3, I4, I5, I6, I7;
input [1:0] s;
reg out;
always @(I0, I1, I2, I3, I4, I5, I6, I7)
case (s)
3'b000 : out = I0;
3'b001 : out = I1;
3'b010 : out = I2;
3'b011 : out = I3;
3'b100 : out = I4;
3'b101 : out = I5;
3'b110 : out = I6;
3'b111 : out = I7;
```

8 : 1 MUX



Truth table

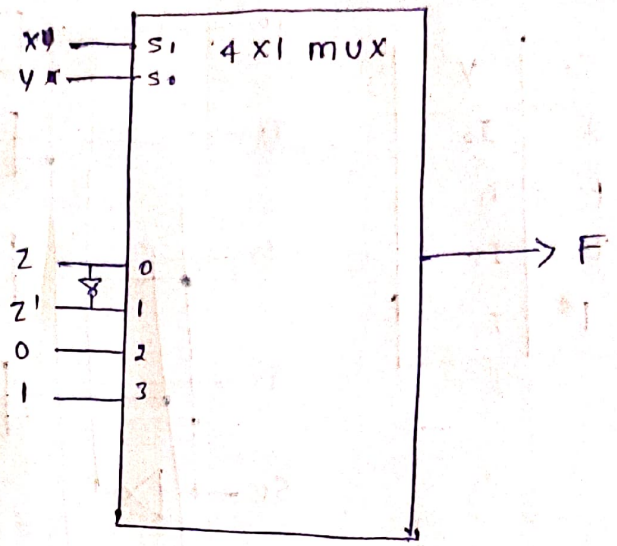
S2	S1	S0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

Boolean function implementation

$F(x,y,z) = \sum(1,2,6,7)$

	Inputs x	y	z		F	
0	0	0	0	0	0	F = z
1	0	0	1		1	
2	0	1	0	1	1	F = z'
3	0	1	1		0	
4	1	0	0	2	0	F = 0
5	1	0	1		0	
6	1	1	0	3	1	F = 1
7	1	1	1		1	

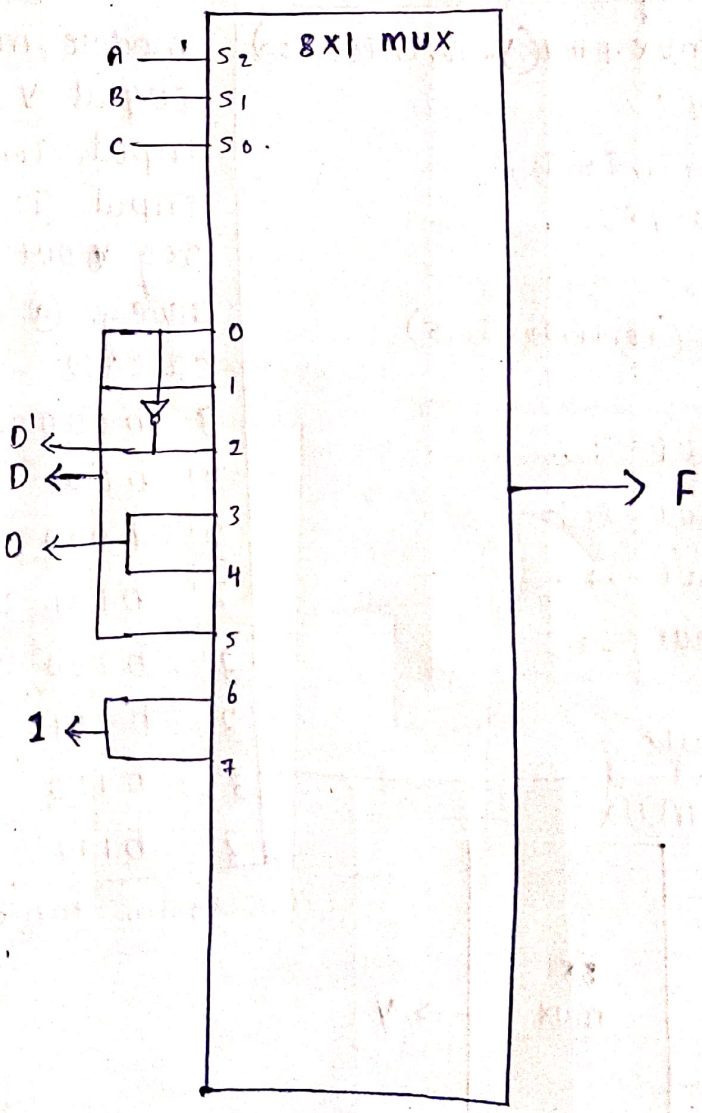
multiplexer implementation



$F(A,B,C,D) = \sum(1,3,4,11,12,13,14,15)$

	A	B	C	D		F	
0	0	0	0	0	0	0	F = D
1	0	0	0	1		1	
2	0	0	1	0	1	0	F = D
3	0	0	1	1		1	
4	0	1	0	0	2	1	F = D'
5	0	1	0	1		0	
6	0	1	1	0	3	0	F = 0
7	0	1	1	1		0	
8	1	0	0	0	4	0	F = 0
9	1	0	0	1		0	
10	1	0	1	0	5	0	F = D
11	1	0	1	1		1	
12	1	1	0	0	6	1	F = 1
13	1	1	0	1		1	
14	1	1	1	0	7	1	F = 1
15	1	1	1	1		1	

multiplexer implementation

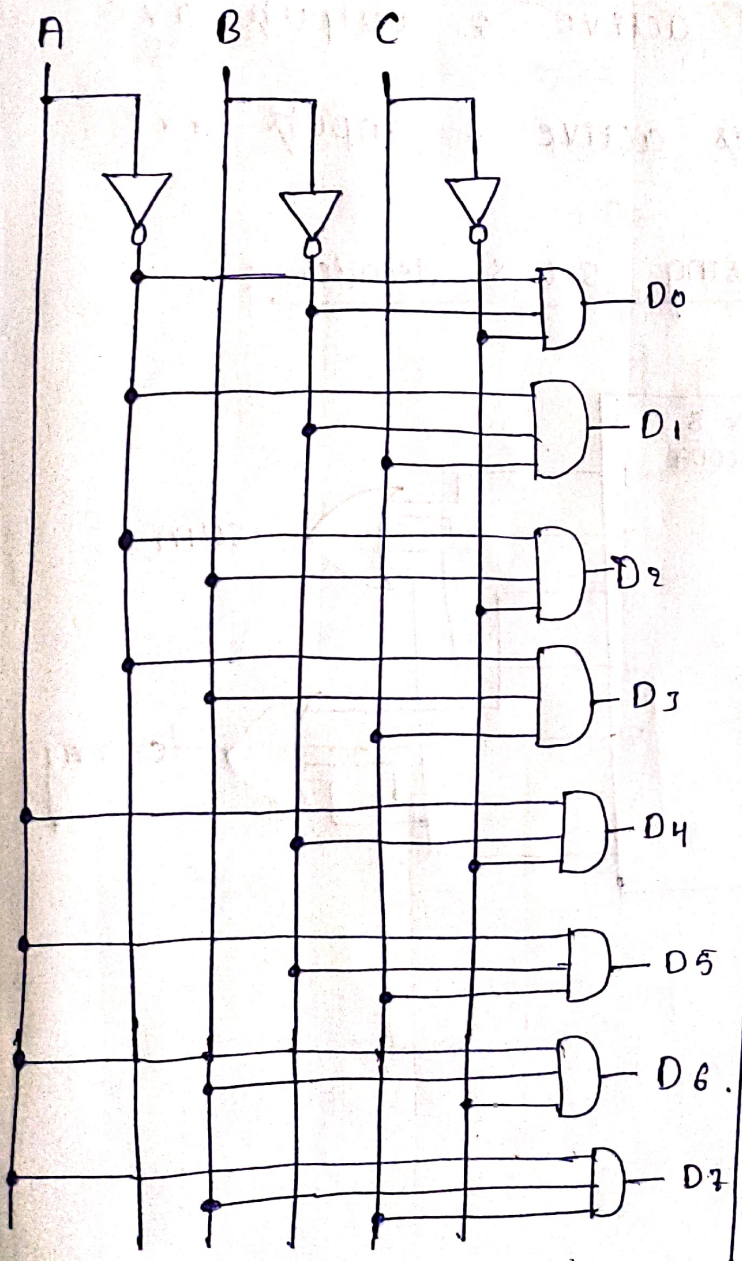


DECODER: DECODER is a combinational circuit that converts a binary information from n i/p to a 2^n unique outputs

THREE to EIGHT LINE DECODER

I/P		
A	B	C
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

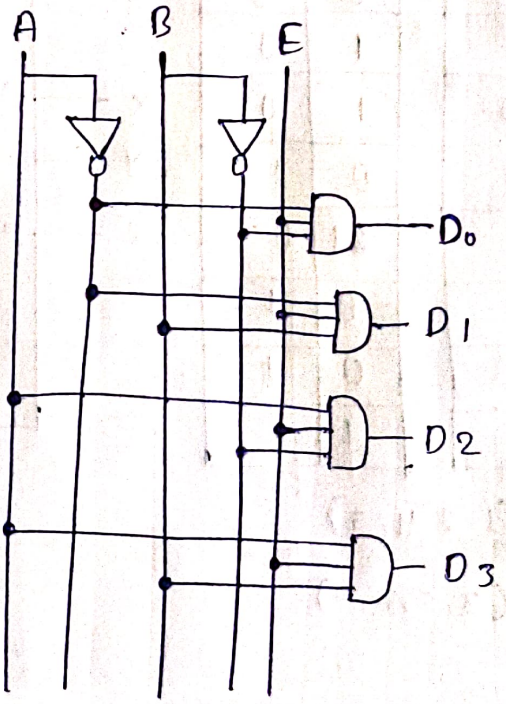
O/P							
$A'B'C'$	$A'B'C$	$A'BC'$	$A'BC$	$AB'C'$	$AB'C$	ABC'	ABC
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	1



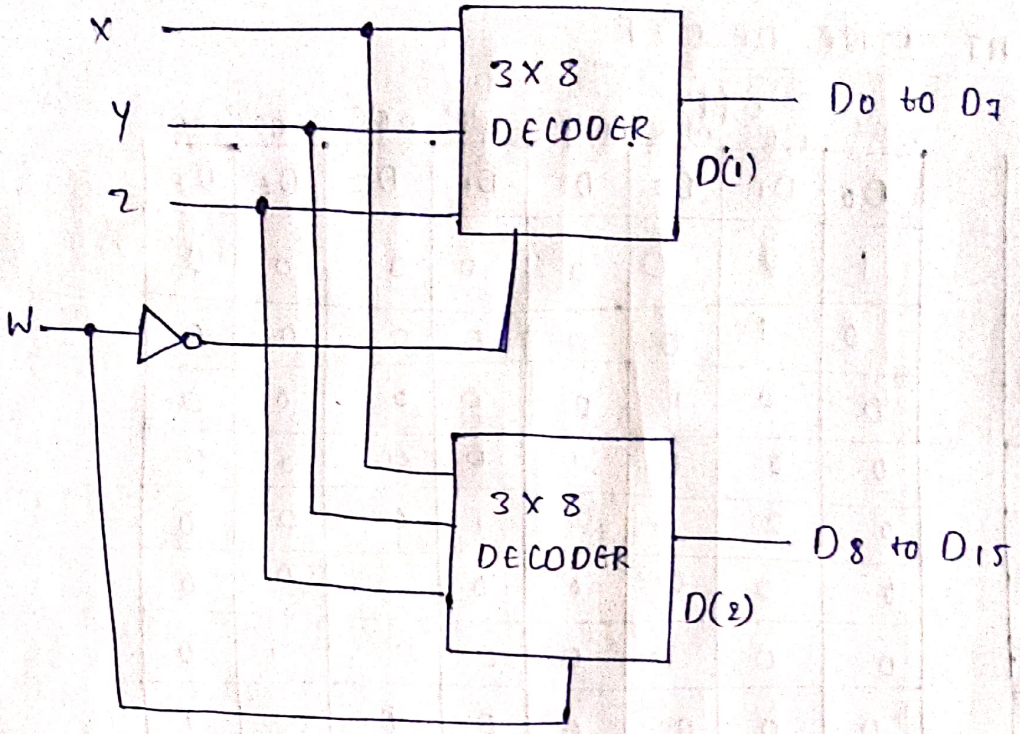
2 to 4 line decoder using AND

E	A	B
1	X	X
0	0	0
0	0	1
0	1	0
0	1	1

$A'B'$	$A'B$	AB'	AB
D_0	D_1	D_2	D_3
1	1	1	1
0	1	1	1
1	0	1	1
1	1	0	1
1	1	1	1



4 x 16 DECODER USING 3 x 8 DECODER



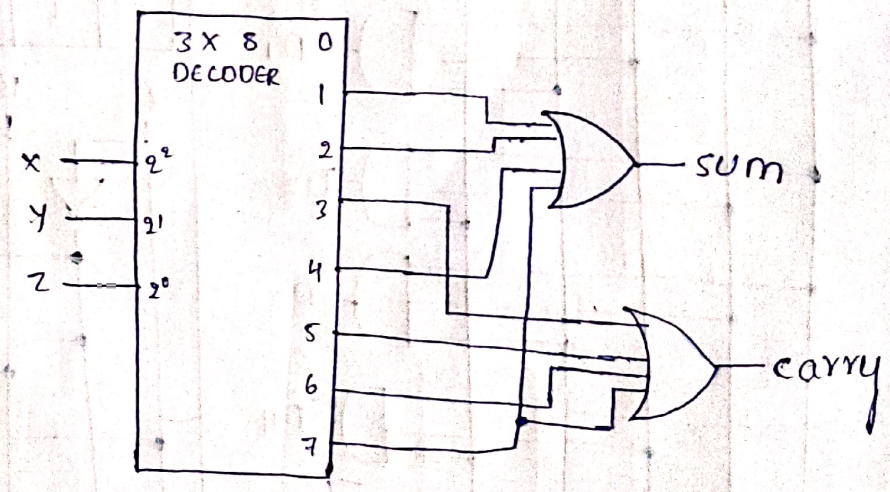
When, W = 0, 1st decoder D(1) is active & outputs are D0 to D7

When, W = 1 2nd decoder D(2) is active & inputs are D8 to D15

Implementation of full adder using 3 to 8 decoder:

	X	Y	Z
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

SUM	CARRY
0	0
1	0
1	0
0	1
1	0
0	1
0	1
1	1



$SUM = \Sigma(1, 2, 4, 7)$

$CARRY = \Sigma(3, 5, 6, 7)$

Encoder:

Octal to binary decoder:

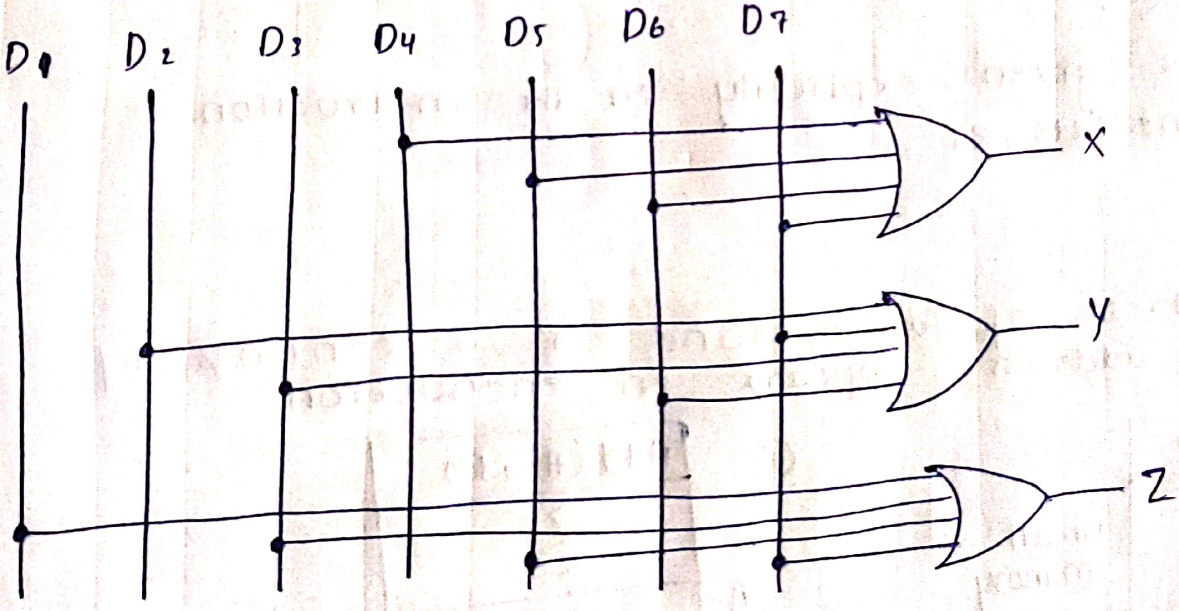
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0
0	0	0	0	0	1	0	0
0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	1

	X	Y	Z
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

$X = D_4 + D_5 + D_6 + D_7$

$Y = D_2 + D_3 + D_6 + D_7$

$Z = D_1 + D_3 + D_5 + D_7$



MODULE-03

1) ADDRESSING MODE :

The different way in which the location of a operand is specified in a instructions are called addressing mode

TYPES OF AM

1) Register addressing mode
2) Absolute mode
3) immediate mode
4) indirect mode
5) index mode
6) Relative addressing mode
7) Auto decrement and increment mode.

1) Register addressing mode :

* operand is a register.
* the operand is a memory location

Eg: MOVE R2, R1
EA = R1

2) Absolute mode :

the address of the memory location is given explicitly in the instructions.

* the operand is a memory location

Eg: move LOCA, R2

3) Immediate mode :

the operand is given explicitly in the instruction

Eg: move 200 immediate, R0
move #200, R0

4) Indirect mode :

the effective address of a operand is a content of a register whose address appears in instruction.

Eg: ① A | Add (R1), R0 | ... | B | OPERAND — main memory.

through general purpose of register

② Add (A), R0.
A | B | ... | B | OPERAND

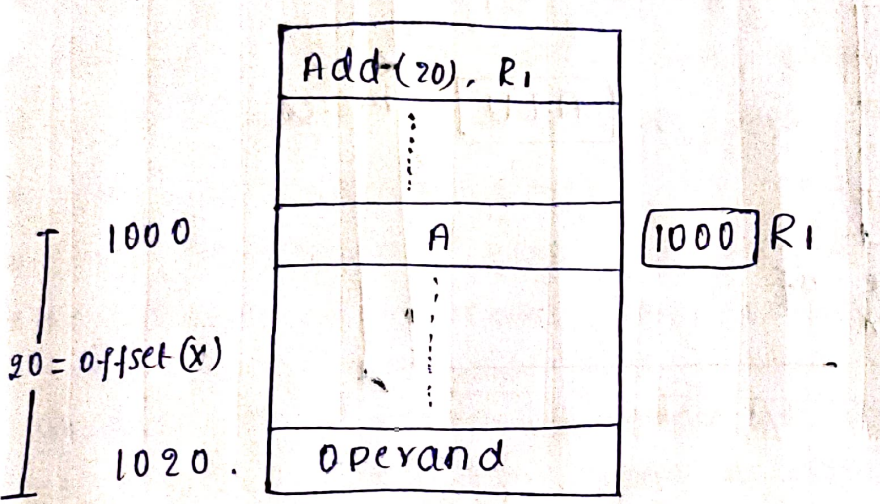
through main memory.

⑤ Index mode :

the effective address of a operand generating a constant value ~~en~~ to contents of a register.

Eg : * X [Ri]

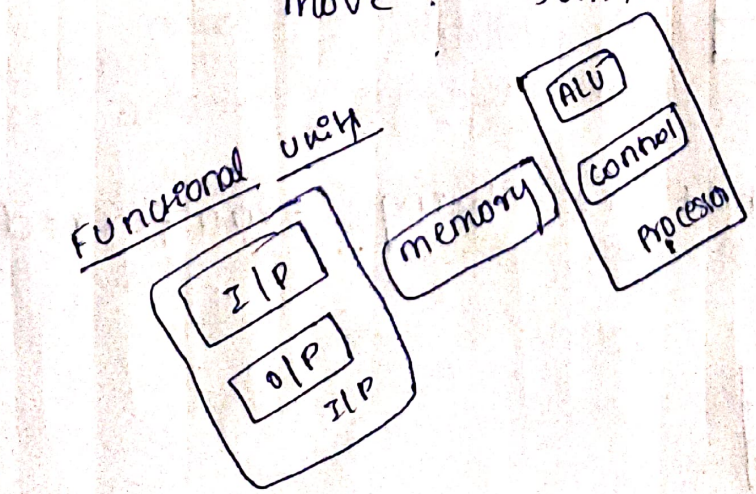
* EA = X + Ri
* Ri
* X



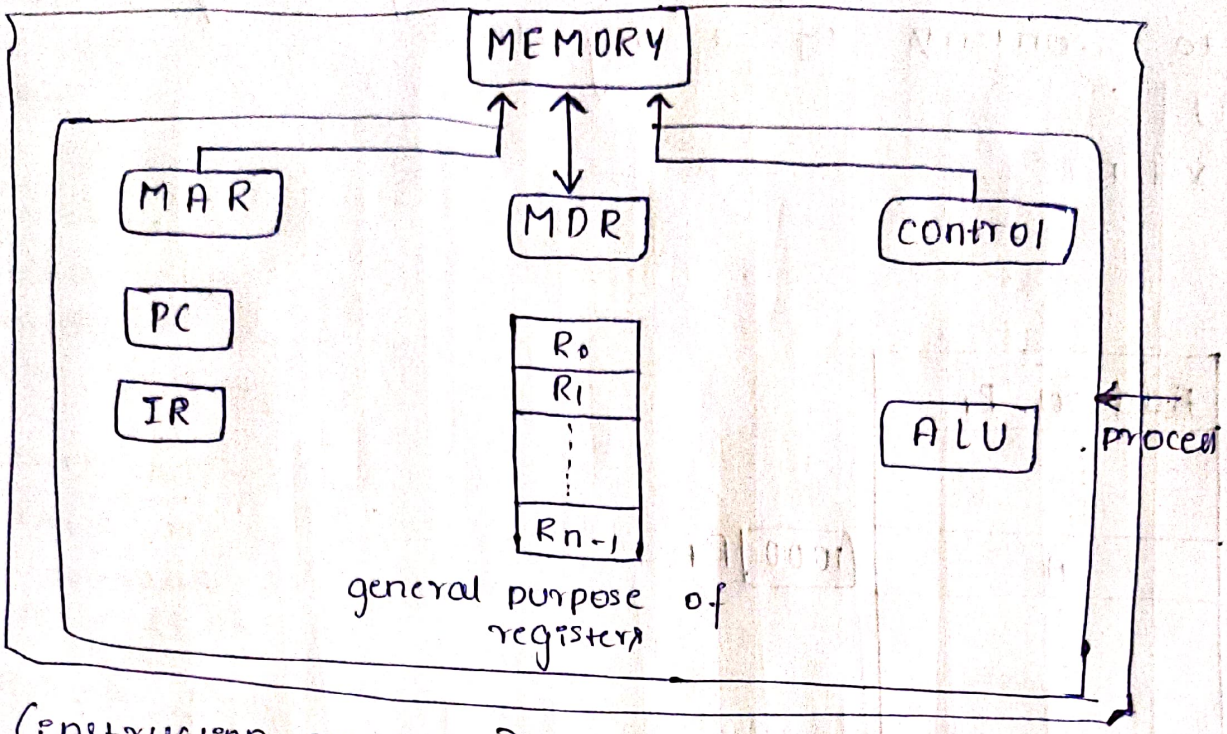
⑥ Relative addressing mode

Address	contents.		
	Move	N, R1	988
	move	#NUM1, R2	992
	clear	R0.	996
loop	Add	(R2), R0.	1000
	Add	#4, R2	1004
	decrement	R1	1008
	Branch>0.	loop	1012
	move	sum, R0.	1016

} initialization



2) BASIC OPERATIONAL CONCEPT :



IR (instruction register) :

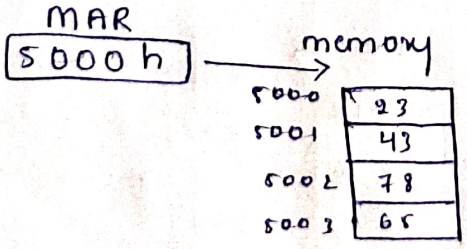
It is a register that contains CPU that temporarily stores the ~~inst~~ instructions to be executed.

PC (Program counter) :

It is a register that contains a adress of a next instructions to be executed.

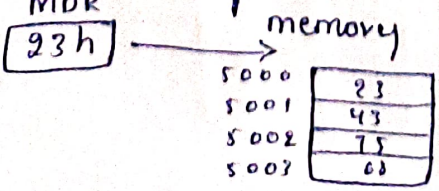
MAR (memory address register) :

it contcens a address of data to be stored in a memory



MDR :

it contains a data that is being transferred to or from memory.



control unit

* it control data transfer operation b/w memory & processor
* ——— 11 ——— 1 ——— 1 ——— 1—— I/O processor.

ALU:

it perform ALU for a given operation.

Explanation:

* The address of a first instruction are stored in PC
* The concept of PC are transferred to MAR
* read signal is sent to memory.
* The first instructions is read out & loaded to MAR.
* The concept of MDR is transferred to IR.
* IR decodes & executes the information.
* To fetch the operand the address is placed into MAR or control unit issues read signal as a result memory is transferred from MAR to MDR to a general purpose of register.
* The address is placed into MAR and data is placed into MDR and control unit issues write signal.
* Result is sent to memory.

3) BASIC INSTRUCTIONS TYPE

(or)

Write one, two, three address instruction format to carryout C ← [A] + [B]

There are 3 types in basic instructions

1) Three address instruction format
2) TWO address instruction format
3) one address instruction format.

⟹

1) Three address instruction format : $C \leftarrow A + B$.

~~Th~~ Consider the arithmetic expression ~~$C \leftarrow [A] + [B]$~~

the steps during evaluation are

* Access the first memory operand denoted by A
* Access the second memory operand that denoted by B
* Perform the addition b/w 2 memory operand.
* Store the result in 3rd memory location C
* The mathematical expression is $C \leftarrow [A] + [B]$

three address instruction format is.

OPcode	source-1	source-2	Destination

Destination = [source-1] + [source-2]

Eg: Add A, B, C

$C \leftarrow [A] + [B]$

2) Two address instruction format :

the format is.

OPcode	source	source/Destination

Destination = [source] + [Destination]

The sequence for instruction is.

move A, R_0

move B, R_1

Add R_0, R_1

move R_1, C

3) one address instruction format :

the format is

OP code	operand

Eg : 1) load A => this loads the content of A into accumulator

2) store A => this stores the accumulator value into memory location.

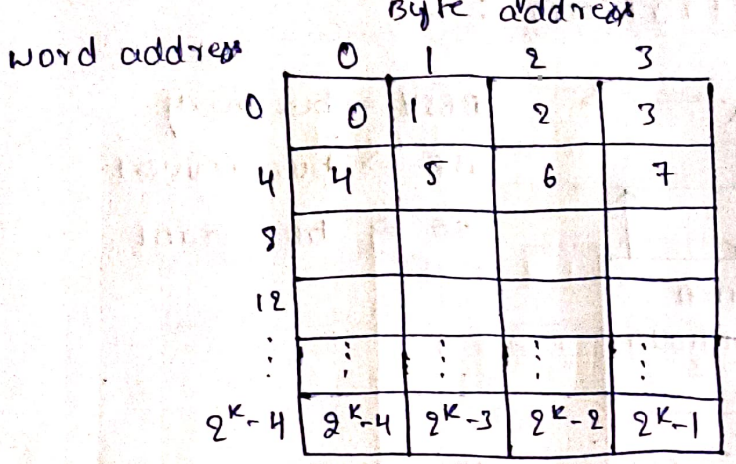
3) Add B => this adds value stored previously with B & stores in accumulator

Acc $\leftarrow$ [Acc] + [B]

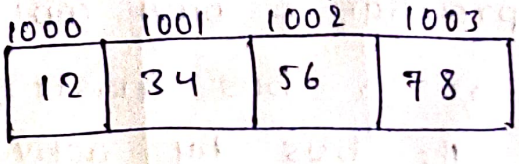
④ BIG ENDIAN :

lowest byte stored in highest address and highest byte stored in lowest address.

Representation of Big endian :



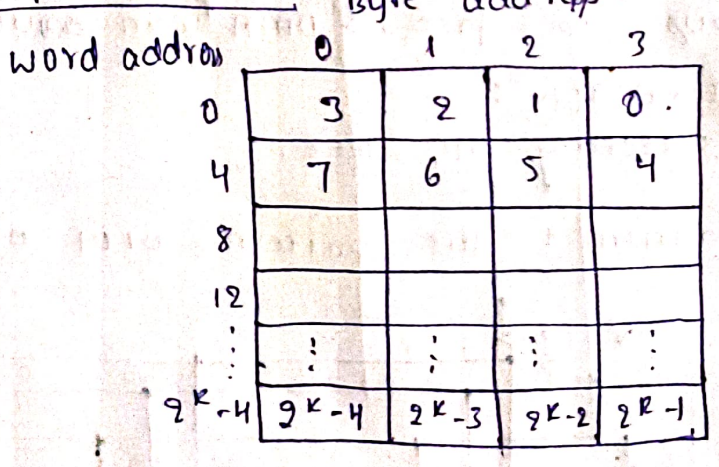
Eg: 0X12345678



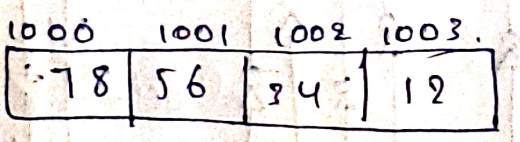
LITTLE ENDIAN :

Highest byte stored in highest address and lowest byte stored in lowest address.

Representation :



Eg: 0X12345678

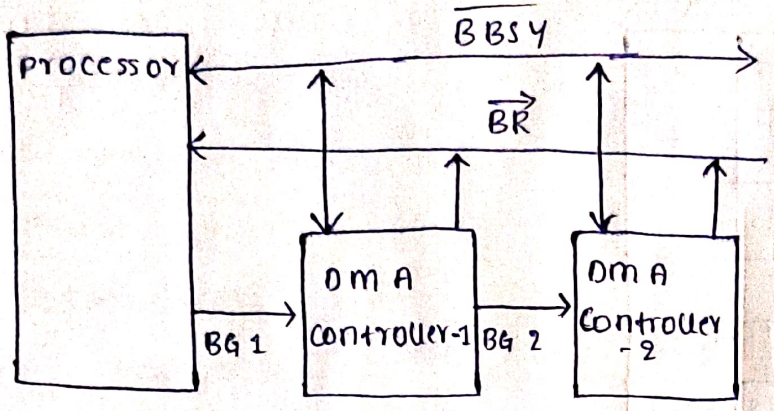


MODULE - 04

1) BUS ARBITRATION :

The process of selecting a bus master is called as bus arbitration.

1) centralized bus arbitration :

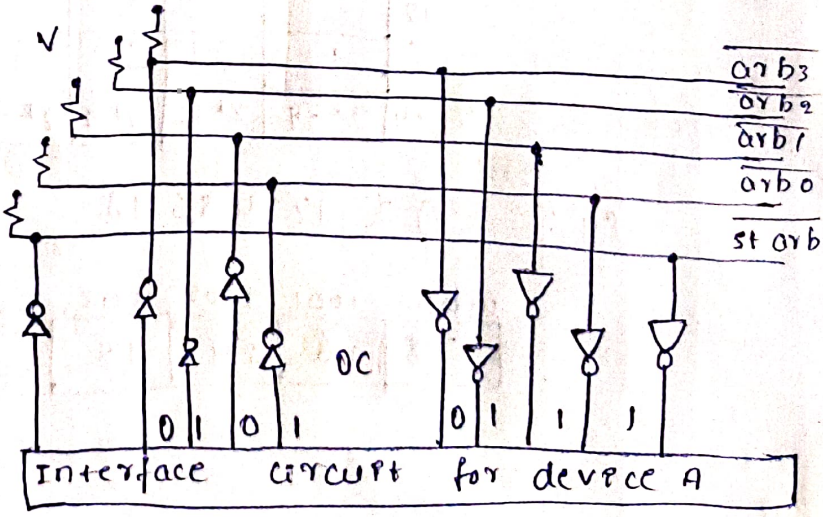


$\overline{BBSY}$ ⇒ bus busy
$\overline{BR}$ ⇒ bus request
BG ⇒ bus grant.

* A single bus arbitar performs the bus arbitration
* The arbitar may be processor & DMA controller.
* normally, processor acts as a bus master.
* DMA controller require to control the bus by activating BR line.
* In response, processor activater the BG1 line.
* BG1 line is connected to all DMA controller and daisy chain fashion.
* When BBSY is 0 implies that bus is busy, and when BBSY is 1 implies that bus is free, & DMA can acquire the bus

2) DISTRUBUTED BUS ARBITRATION :

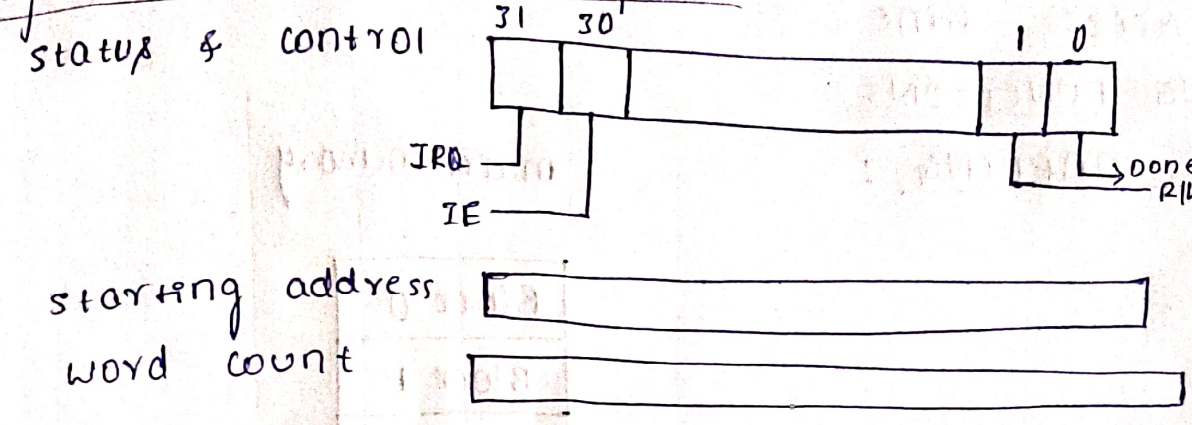
* All device participate in selecting bus free.
* For a two devices A & B.
* device A has ID 5 so it transmit the pattern 0101 on a arbitration lines.
* device B has ID 6 so it transmit a patter 0110 on a arbitration lines.
* The pattern that appears on a arbitration lines. is a logical OR of a pattern. and the pattern should be 0111 on a arbitration lines.



2) DIRECT

The process of transferring a lot of data from I/O device and main memory without intervention process are called DMA.

Fig: Register in DMA interface



Register in DMA controller.

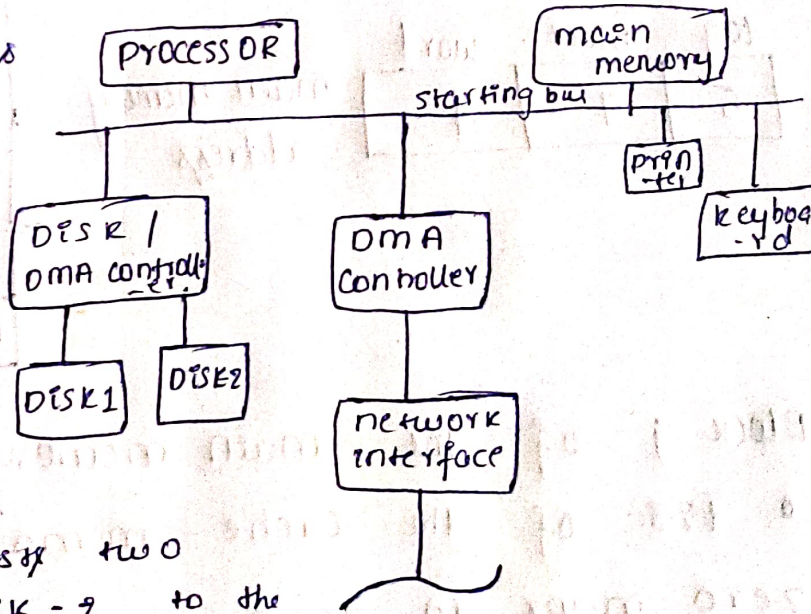
Types of register

1) starting address : it is used to store the ~~RFO~~ starting address of memory block.

2) status & control : It consists of done bit, Read/write bit, IRQ & IE-bit

3) word count : It is used to store the number of words to be transferred from main memory.

Fig: computer with DMA controller:



* The processor initiate the bus transfer
* To initiate the bus transfer. the processor must enforms the DMA ~~controller~~ transfer
  i) starting address
  ii) no of words in a block.
  iii) Direction to transfer.
* The DMA controller consists two external devices disk-1 & disk-2 to the system bus as shown in the above figure.

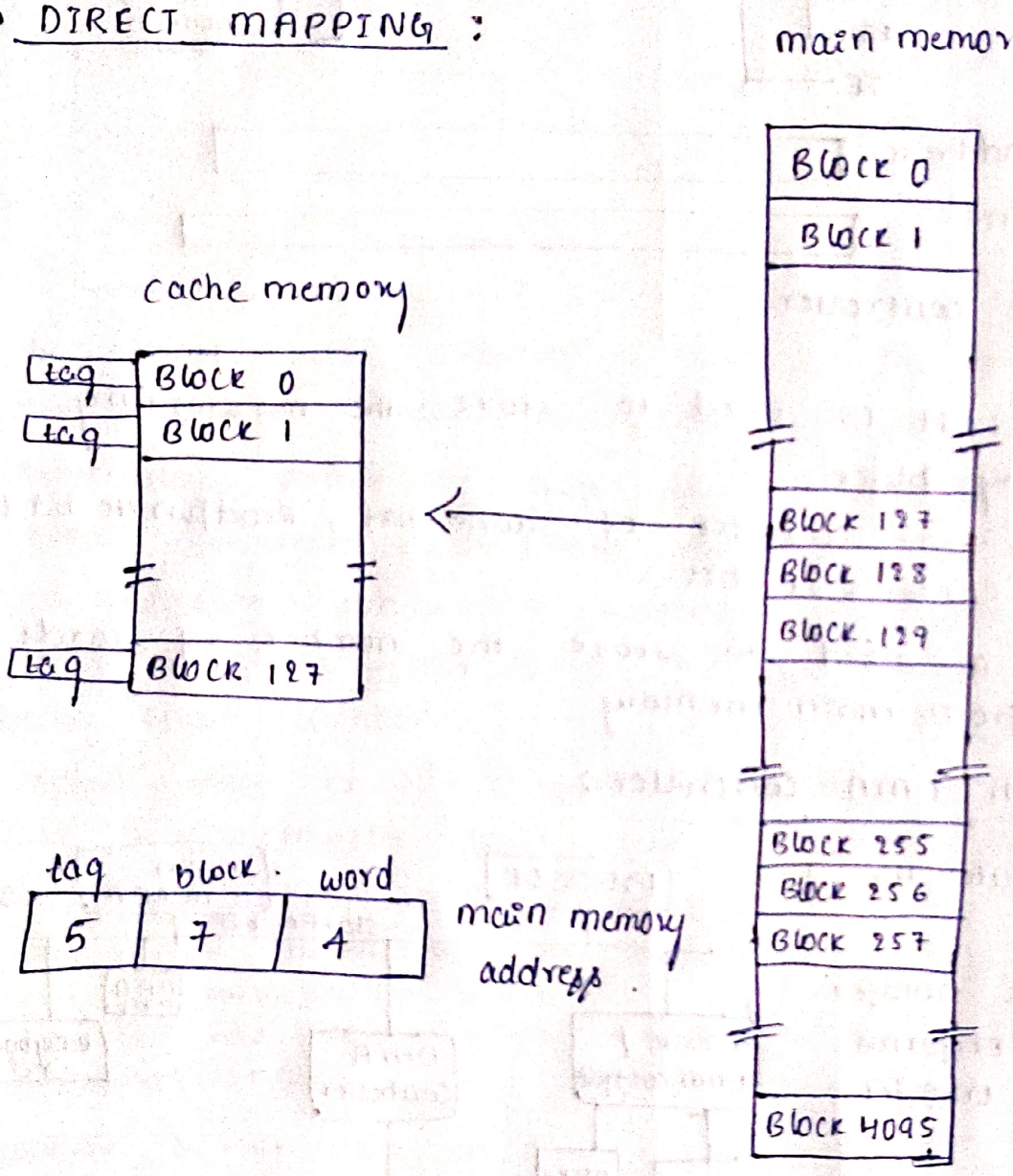
3) What is cache memory ? Explain mapping function.

1) Cache memory :

Cache memory is a fast access memory located in b/w a processor and main memory. TO designed the reduce access time.

MAPPING FUNCTIONS :

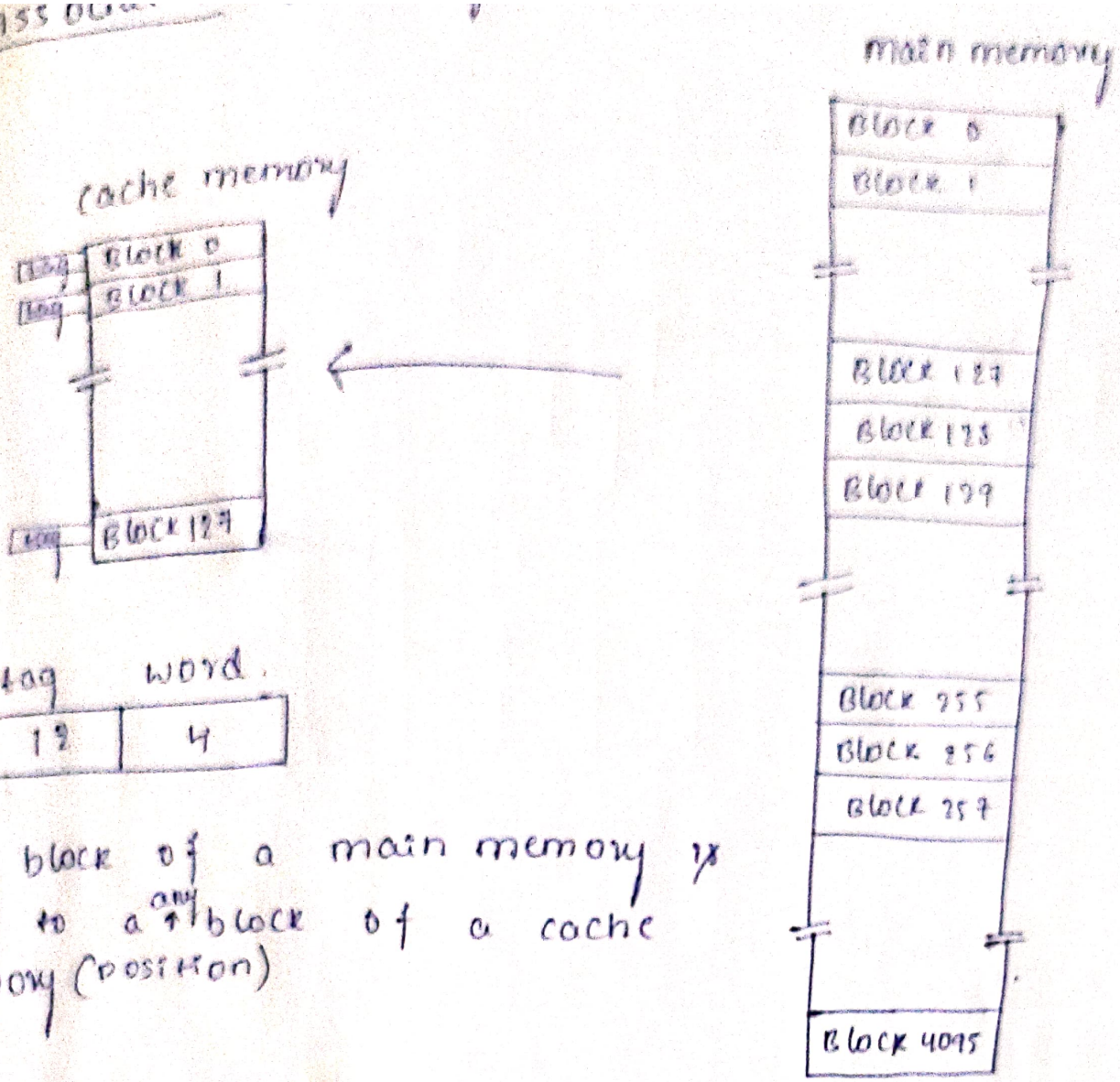
1) DIRECT MAPPING :



Block j of the main memory is maps to a ; modulo 128 of the cache memory

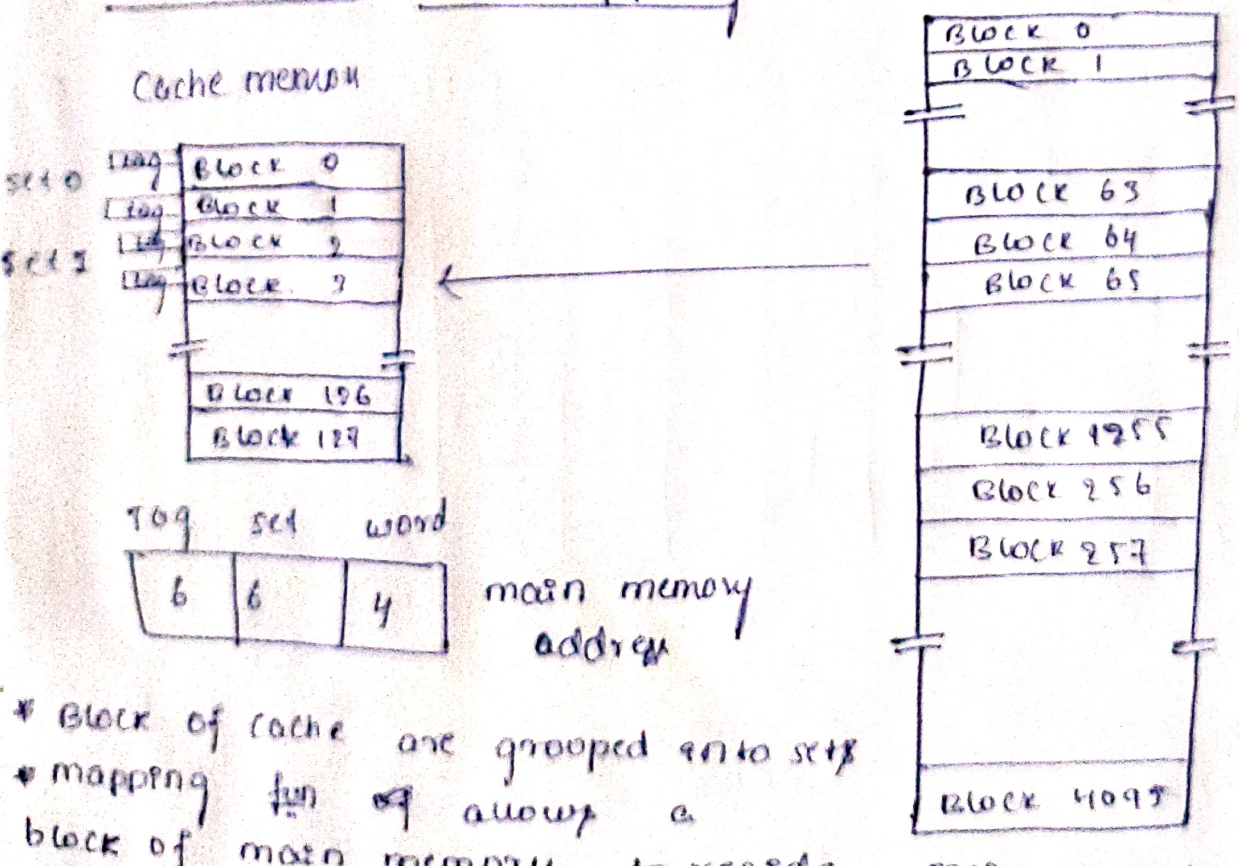
zero maps to zero. and 129 maps to one.

ii) Associ



Any block of a main memory is maps to a any block of a cache memory (position)

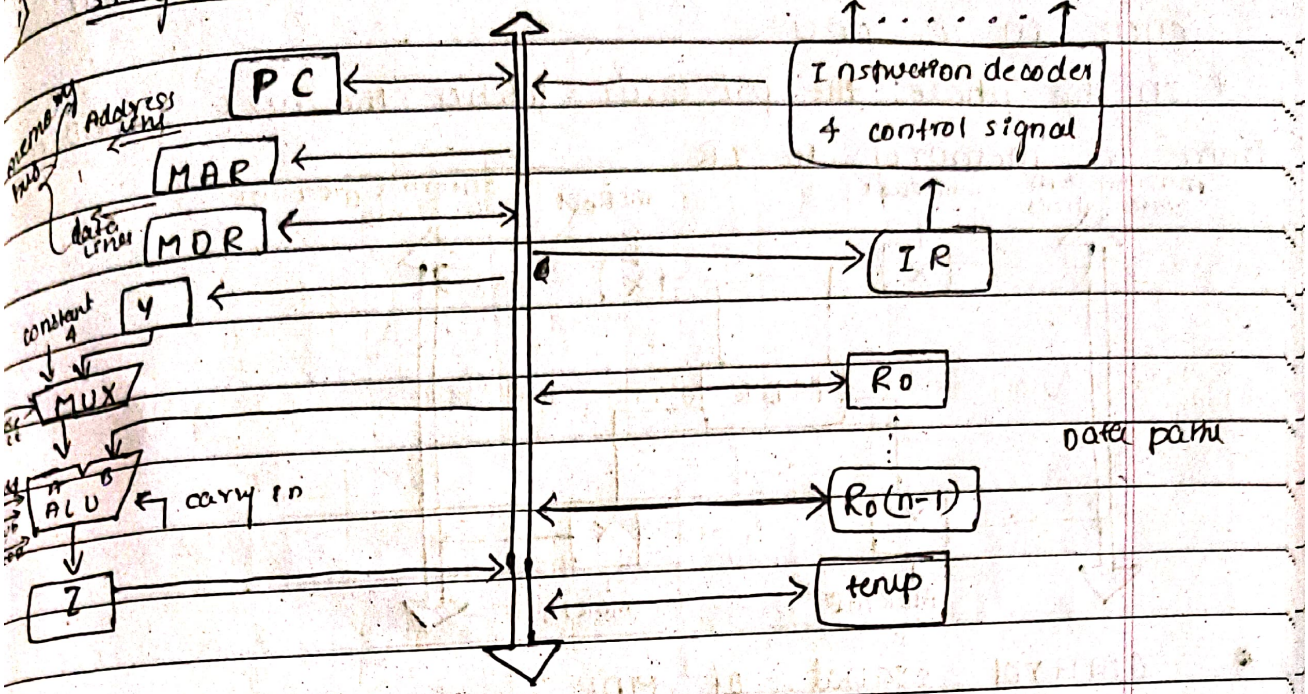
iii) set-Associative mapping:



* Block of cache are grouped into sets
* mapping fun allows a block of main memory to reside in any block of specific set.

MODULE - 05 :

1) Single bus structure :

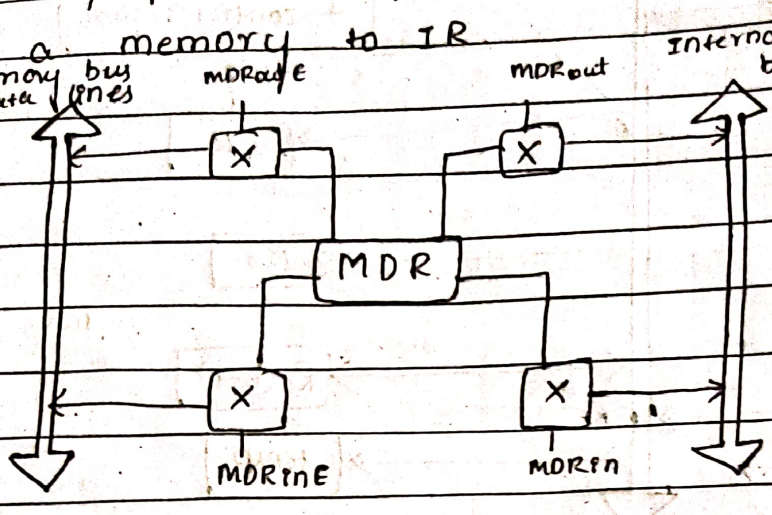


* The address of the 1st instruction are stored in PC
* The content of PC are transferred to MAR
* read signal is sent to memory
* The first instruction is read out & loaded to MAR
* The content of MDR is transferred to IR
* IR decodes & executes the information.
* To fetch the operands its address is placed into MAR & control unit issues read signal, as a result operand is transferred from memory into MDR & MAR ~~send~~ to a general purpose register and transferred to ALU it perform some operations.
* The address is placed into MAR and data is placed into MDR and control unit issues write signal
* Result is stored in memory.
* mux are having 2 inputs
  - → output of Y
  - → constant value 4.
* Transfer ~~data~~ word of data from one register to another.
* store word of data from register to a given memory location.

2) Explain the fetch operation

⇒ * Fetch operation is a first phase of the instruction cycle in a CPU

* In this phase, the processor fetches the next instruction from a memory to IR



* Control signal of MDR.

* MDRin and MDRout controls the connection of a internal processor data bus

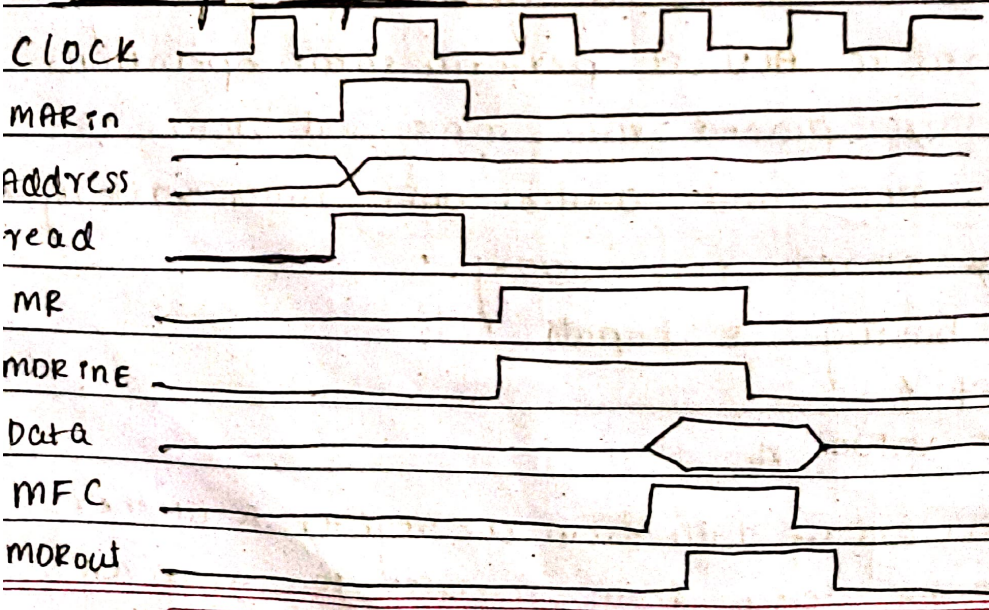
* MDRinE & MDRoutE controls the connection of a external processor data bus

semiary.

* MARin & ~~MARout~~ controls the connection of a internal processor addressing bus.

* MARout controls the connection of a memory address bus

Timing Diagram :



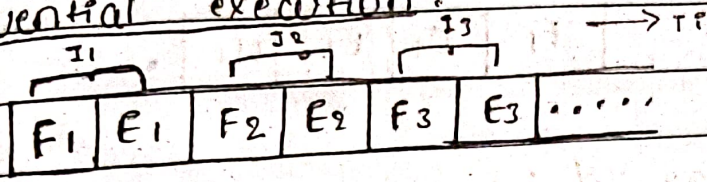
DATE / /
PAGE NO.

3) Pipelining : Dividing the instruction into a number of stages and execute them in parallel are called pipelining.

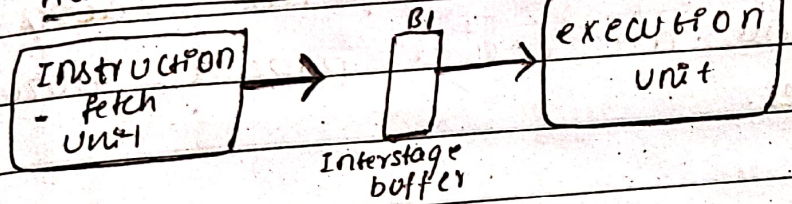
Two stage pipelining :

Here instructions are divided into two Fetch & execute.

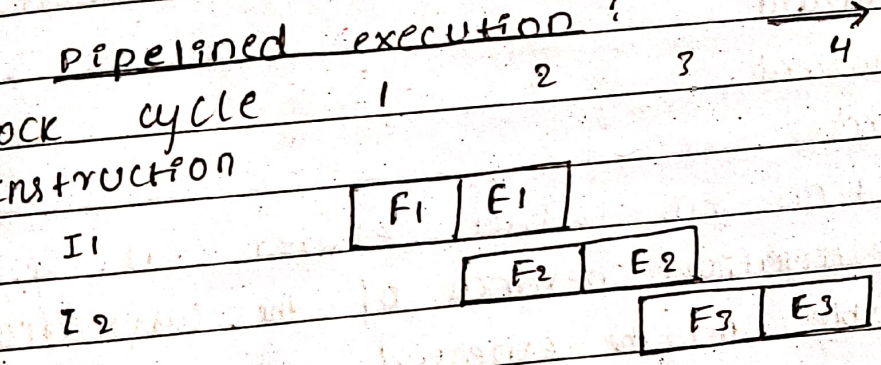
a) sequential execution :



b) Hardware ~~execution~~ organisation :



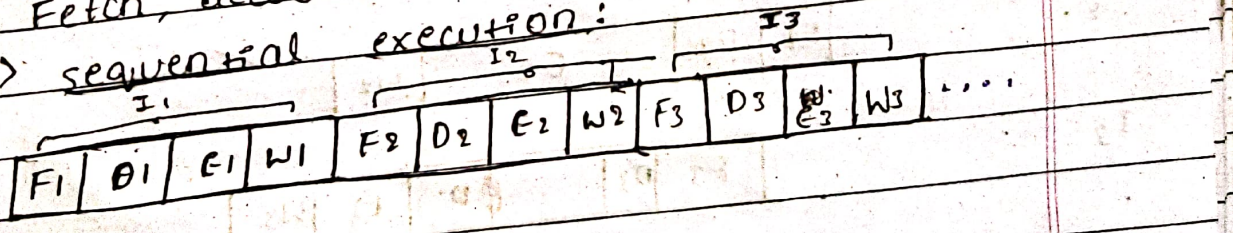
c) Pipelined execution :



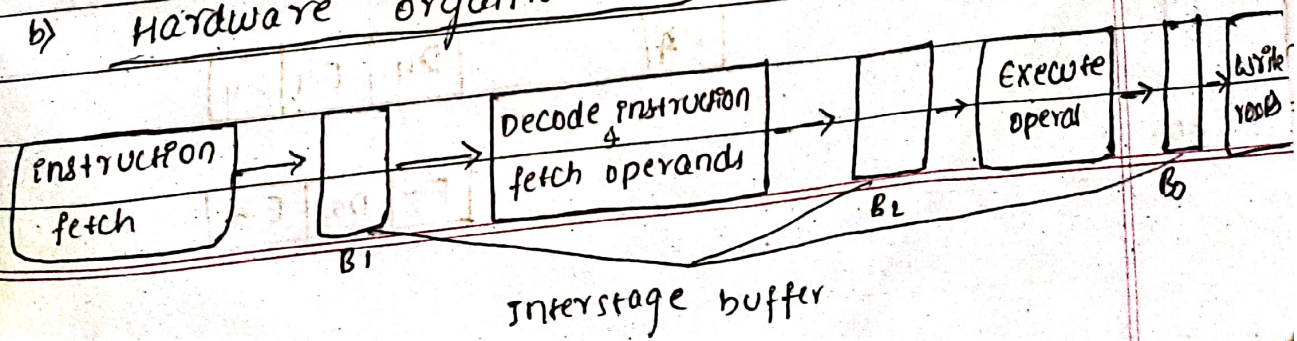
FOUR STAGE PIPELINING :

Here instruction are divided into 4 stages Fetch, decode, execute & write

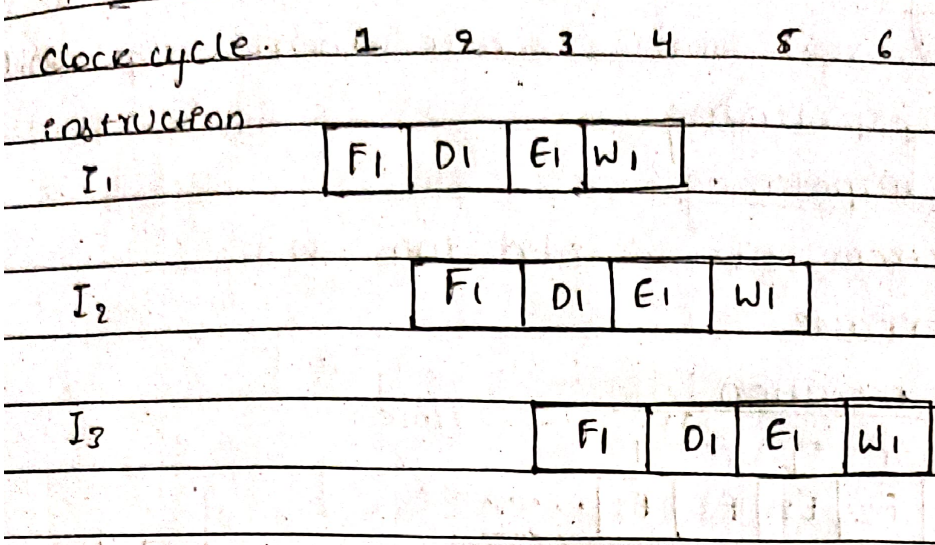
a) sequential execution :



b) Hardware organisation :



b) Pipelined execution → time



4) Pipeline Hazard :

Any condition that ~~pipeline~~ causes Pipeline to stall are called pipeline Hazard

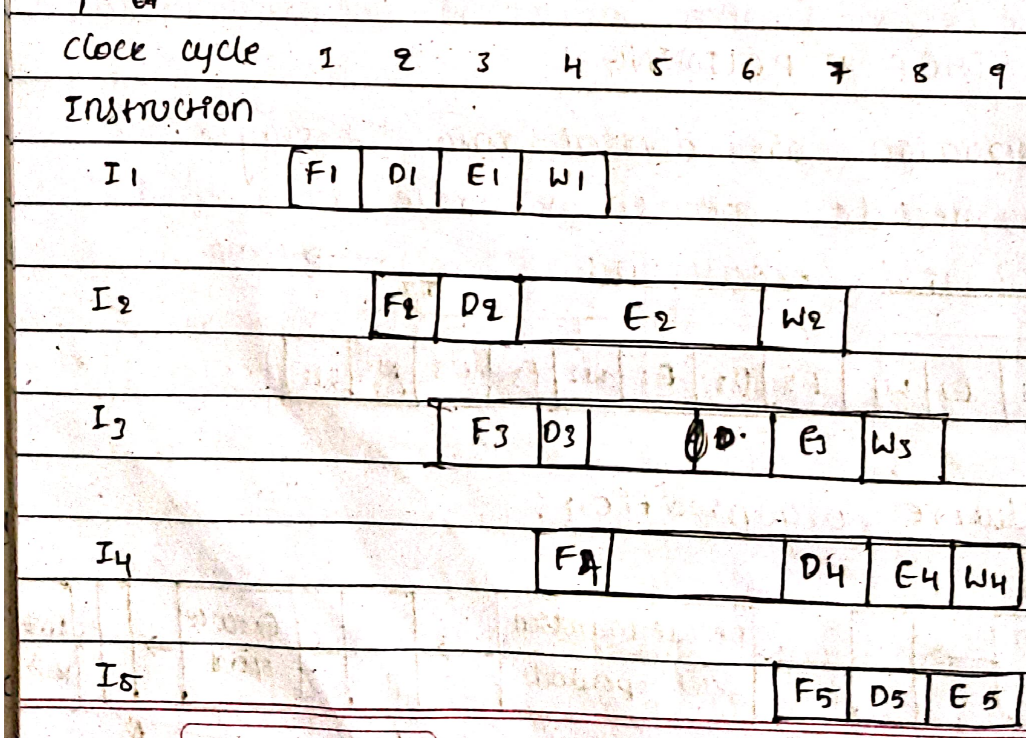
It has 3 types.

* Data HAZARD
* Instruction HAZARD
* Structural Hazard.

i) Data HAZARD :

Any condition in which either in a source or destination operands of the instruction are not available on the expected time in pipeline

Eg:

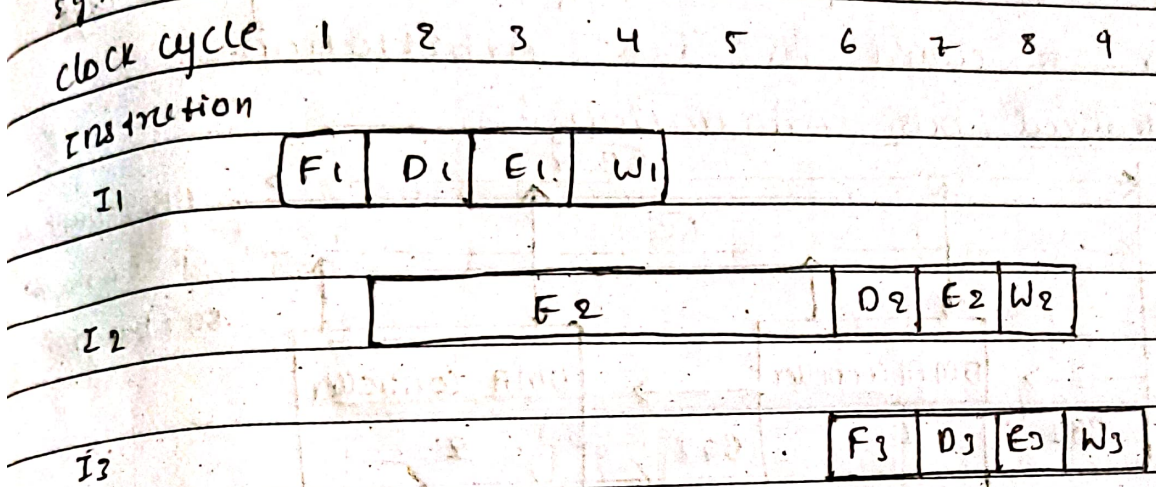


DATE / /
PAGE NO.

2) Instruction Hazard :

this pipeline is stall because of availability of instruction

Eg:



3) Structural Hazard :

due to availability of resources

Eg: